```python
# Participating students:
# Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran

# python3 -m pip install tabulate
from tabulate import tabulate
import BitVector

def to_matrix(byte_list):
    """Convert a flat list of 16 bytes into a 4x4 column-major matrix."""
    matrix_byte = [[0]*4 for _ in range(4)]
    for col in range(4):
        for row in range(4):
            matrix_byte[row][col] = byte_list[col * 4 + row]
    return matrix_byte

def xor_matrix(a,b):
    """XOR two 4x4 matrices element-wise."""
    new_matrix = [[0]*4 for _ in range(4)]
    for col in range(4):
        for row in range(4):
            new_matrix[row][col] = a[row][col] ^ b[row][col]
    return new_matrix

# S-box table read row by row from the image (row = high nibble, col = low nibble)
_SBOX_ROWS = [
    [0x63,0x7C,0x77,0x7B,0xF2,0x6B,0x6F,0xC5,0x30,0x01,0x67,0x2B,0xFE,0xD7,0xAB,0x76], # 0x
    [0xCA,0x82,0xC9,0x7D,0xFA,0x59,0x47,0xF0,0xAD,0xD4,0xA2,0xAF,0x9C,0xA4,0x72,0xC0], # 1x
    [0xB7,0xFD,0x93,0x26,0x36,0x3F,0xF7,0xCC,0x34,0xA5,0xE5,0xF1,0x71,0xD8,0x31,0x15], # 2x
    [0x04,0xC7,0x23,0xC3,0x18,0x96,0x05,0x9A,0x07,0x12,0x80,0xE2,0xEB,0x27,0xB2,0x75], # 3x
    [0x09,0x83,0x2C,0x1A,0x1B,0x6E,0x5A,0xA0,0x52,0x3B,0xD6,0xB3,0x29,0xE3,0x2F,0x84], # 4x
    [0x53,0xD1,0x00,0xED,0x20,0xFC,0xB1,0x5B,0x6A,0xCB,0xBE,0x39,0x4A,0x4C,0x58,0xCF], # 5x
    [0xD0,0xEF,0xAA,0xFB,0x43,0x4D,0x33,0x85,0x45,0xF9,0x02,0x7F,0x50,0x3C,0x9F,0xA8], # 6x
    [0x51,0xA3,0x40,0x8F,0x92,0x9D,0x38,0xF5,0xBC,0xB6,0xDA,0x21,0x10,0xFF,0xF3,0xD2], # 7x
    [0xCD,0x0C,0x13,0xEC,0x5F,0x97,0x44,0x17,0xC4,0xA7,0x7E,0x3D,0x64,0x5D,0x19,0x73], # 8x
    [0x60,0x81,0x4F,0xDC,0x22,0x2A,0x90,0x88,0x46,0xEE,0xB8,0x14,0xDE,0x5E,0x0B,0xDB], # 9x
    [0xE0,0x32,0x3A,0x0A,0x49,0x06,0x24,0x5C,0xC2,0xD3,0xAC,0x62,0x91,0x95,0xE4,0x79], # Ax
    [0xE7,0xC8,0x37,0x6D,0x8D,0xD5,0x4E,0xA9,0x6C,0x56,0xF4,0xEA,0x65,0x7A,0xAE,0x08], # Bx
    [0xBA,0x78,0x25,0x2E,0x1C,0xA6,0xB4,0xC6,0xE8,0xDD,0x74,0x1F,0x4B,0xBD,0x8B,0x8A], # Cx
    [0x70,0x3E,0xB5,0x66,0x48,0x03,0xF6,0x0E,0x61,0x35,0x57,0xB9,0x86,0xC1,0x1D,0x9E], # Dx
    [0xE1,0xF8,0x98,0x11,0x69,0xD9,0x8E,0x94,0x9B,0x1E,0x87,0xE9,0xCE,0x55,0x28,0xDF], # Ex
    [0x8C,0xA1,0x89,0x0D,0xBF,0xE6,0x42,0x68,0x41,0x99,0x2D,0x0F,0xB0,0x54,0xBB,0x16], # Fx
]

def substitute(byte):
    """Look up a single byte in the AES S-box."""
    row = (byte & 0xF0) >> 4  # high nibble selects the row
    col = (byte & 0x0F)       # low nibble selects the column
    return _SBOX_ROWS[row][col]

def next_key(key_matrix, rcon):
    """Compute the next round key using AES key schedule."""
    # Pull out columns
    W0 = [key_matrix[r][0] for r in range(4)]
    W1 = [key_matrix[r][1] for r in range(4)]
    W2 = [key_matrix[r][2] for r in range(4)]
    W3 = [key_matrix[r][3] for r in range(4)]

    # W3 = [a,b,c,d] -> rotate -> [b,c,d,a] -> SubBytes -> XOR rcon into first byte
    a, b, c, d = W3
    transform = [substitute(b) ^ rcon, substitute(c), substitute(d), substitute(a)]

    W4 = [W0[r] ^ transform[r] for r in range(4)]
    W5 = [W1[r] ^ W4[r]        for r in range(4)]
    W6 = [W2[r] ^ W5[r]        for r in range(4)]
    W7 = [W3[r] ^ W6[r]        for r in range(4)]

    return [[W4[r], W5[r], W6[r], W7[r]] for r in range(4)]

def sub_bytes(matrix):
    """Apply the AES S-box substitution to every byte in the 4x4 matrix."""
    result = [[0]*4 for _ in range(4)]
    for row in range(4):
        for col in range(4):
            result[row][col] = substitute(matrix[row][col])
    return result

def shift_rows(matrix):
    """Shift each row in the 4x4 matrix row index place to the left"""
    result = [[0]*4 for _ in range(4)]
    for row in range(4):
        for col in range(4):
            # Each row is cyclically shifted left by 'row' positions
            result[row][col] = matrix[row][(col + row) % 4]
    return result

def mix_columns(matrix):
    """Multiplies the 4x4 matrix by the D matrix, and reduces appropriately"""
    result = [[0]*4 for _ in range(4)]
    D = [[0x02, 0x03, 0x01, 0x01],
        [0x01, 0x02, 0x03, 0x01],
        [0x01, 0x01, 0x02, 0x03],
        [0x03, 0x01, 0x01, 0x02]]
    # Polynomial we mod by, represented as a binary string
    mod =  BitVector.BitVector(bitstring = "100011011")
    # Loop through each element in each matrix
    for i in range(4):
        for j in range(4):
            for k in range(4):
                # Multiply values in each row/col pair, and XOR sum
                row_bits = BitVector.BitVector(intVal = D[i][k], size = 8)
                col_bits = BitVector.BitVector(intVal = matrix[k][j], size = 8)
                result[i][j] ^= int(row_bits.gf_multiply(col_bits))
            # Reduce each entry to be within field specified by mod
            element = BitVector.BitVector(intVal = result[i][j])
            # Note gf_divide_by_modulus returns a (quotient, remainder) tuple
            q_r = element.gf_divide_by_modulus(mod, 8)
            result[i][j] = int(q_r[1])
    return result

def fmt_matrix(m):
    """Format a 4x4 matrix as a nice grid with hex values."""
    return tabulate([[f"{x:02X}" for x in row] for row in m], tablefmt="grid")

def main():
    plaintext = [0x49, 0x20, 0x6E, 0x65, 0x65, 0x64, 0x20, 0x61,
                 0x6E, 0x20, 0x41, 0x2B, 0x20, 0x70, 0x6C, 0x7A]
    key0 = [0x45, 0x6E, 0x63, 0x72, 0x79, 0x70, 0x74, 0x20,
           0x6D, 0x79, 0x20, 0x6D, 0x61, 0x72, 0x6B, 0x73]

    msg_matrix  = to_matrix(plaintext)
    key_matrix  = to_matrix(key0)
    key1_matrix = next_key(key_matrix, rcon=0x01)
```

```
    # Round 1 steps
    after_xor   = xor_matrix(msg_matrix, key_matrix)   # 3a. AddRoundKey K0
    after_sub   = sub_bytes(after_xor)                  # 3b. SubBytes
    after_shift = shift_rows(after_sub)                 # 3c. ShiftRows
    after_mix = mix_columns(after_shift)                # 3d. MixColumns
    after_xor_2 = xor_matrix(after_mix, key1_matrix)    # 3e. AddRoundKey K1

    with open("Project5.txt", "w") as f:
        f.write(f"Project 5: Cryptography MATH 4175\n")
        f.write(f"Participating Students:\n")
        f.write(f"Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran\n")

        f.write(f"\nQuestions 1-2:\n\n")
        f.write(f"Selected Plaintext is 49 20 6E 65 65 64 20 61 6E 20 41 2B 20 70 6C 7A\n\n")
        f.write(f"Selected Key0 is 45 6E 63 72 79 70 74 20 6D 79 20 6D 61 72 6B 73\n\n")
        f.write(f"Selected Key1 is 04 11 EC 9D 7D 61 98 BD 10 18 B8 D0 71 6A D3 A3\n\n")
        f.write("Plaintext Matrix Notation:\n")
        f.write(fmt_matrix(msg_matrix))
        f.write("\n\nKey (K0) Matrix Notation:\n")
        f.write(fmt_matrix(key_matrix))
        f.write("\n\nKey (K1) Matrix Notation:\n")
        f.write(fmt_matrix(key1_matrix))

        f.write("\n\nQuestion 3:\n\n")
        f.write("3a. After XOR (AddRoundKey) K0:\n")
        f.write(fmt_matrix(after_xor))
        f.write("\n\n3b. After SubBytes:\n")
        f.write(fmt_matrix(after_sub))
        f.write("\n\n3c. After ShiftRows:\n")
        f.write(fmt_matrix(after_shift))
        f.write("\n\n3d. After MixColumns:\n")
        f.write(fmt_matrix(after_mix))
        f.write("\n\n3e. After After XOR (AddRoundKey) K1:\n")
        f.write(fmt_matrix(after_xor_2))

if __name__ == "__main__":
    main()
```